# USER INTERFACE DESIGN

Name: Vineeth P
Registration no: 240701589

## EXP - 3

To design and implement three different types of user interfaces (CLI, GUI, and VUI) for renaming a file and compare them based on efficiency, learning curve, error handling, and user satisfaction.

### 1. CLI (Command Line Interface)

#### Description:

The CLI version allows users to rename a file by providing the old and new filenames as command-line arguments.

#### Python Code:

```
import os
import sys

def rename_file(old_name, new_name):
    try:
        os.rename(old_name, new_name)
        print(f"File renamed from {old_name} to {new_name}")
    except FileNotFoundError:
        print(f"Error: {old_name} not found.")
    except Exception as e:
        print(f"An error occurred: {e}")

if __name__ == "__main__":
    if len(sys.argv) != 3:
        print("Usage: python rename_file_cli.py <old_filename> <new_filename>")
    else:
        rename_file(sys.argv[1], sys.argv[2])
```

#### How It Works:

- The script accepts two command-line arguments.
- It uses os.rename() to rename the file.

• Handles FileNotFoundError and general exceptions.
• Displays appropriate success or error messages.

## 2. GUI (Graphical User Interface)

### Description:

The GUI version uses Tkinter to provide a graphical interface where users can enter the old and new filenames and click a button to rename the file.

### Python Code:

```
import tkinter as tk
from tkinter import messagebox
import os

def rename_file():
    old_name = old_filename_entry.get()
    new_name = new_filename_entry.get()

    try:
        os.rename(old_name, new_name)
        messagebox.showinfo("Success", f"File renamed from {old_name} to {new_name}")
    except FileNotFoundError:
        messagebox.showerror("Error", f"File {old_name} not found.")
    except Exception as e:
        messagebox.showerror("Error", f"An error occurred: {e}")

root = tk.Tk()
root.title("File Renamer")

tk.Label(root, text="Old Filename:").grid(row=0, column=0)
tk.Label(root, text="New Filename:").grid(row=1, column=0)

old_filename_entry = tk.Entry(root)
old_filename_entry.grid(row=0, column=1)

new_filename_entry = tk.Entry(root)
new_filename_entry.grid(row=1, column=1)

rename_button = tk.Button(root, text="Rename File", command=rename_file)
rename_button.grid(row=2, columnspan=2)

root.mainloop()
```

### Features:

- User-friendly graphical layout.
- Input fields for old and new filenames.
- Rename button to execute action.
- Pop-up messages for success and errors.

## 3. VUI (Voice User Interface)

### Description:

The VUI version allows users to rename files using voice commands. SpeechRecognition library is used to capture and interpret voice input.

### Python Code:

```
import speech_recognition as sr
import os

def rename_file_from_voice_command(command):
    try:
        words = command.split(" ")
        old_name = words[1]
        new_name = words[3]

        os.rename(old_name, new_name)
        print(f"File renamed from {old_name} to {new_name}")
    except Exception as e:
        print(f"Error: {e}")

def listen_for_command():
    recognizer = sr.Recognizer()
    mic = sr.Microphone()

    print("Listening for command to rename a file...")
    with mic as source:
        recognizer.adjust_for_ambient_noise(source)
        audio = recognizer.listen(source)

    try:
        command = recognizer.recognize_google(audio)
        print(f"Command received: {command}")
        rename_file_from_voice_command(command)
    except sr.UnknownValueError:
        print("Sorry, I couldn't understand the command.")
    except sr.RequestError as e:
```

```
        print(f"Could not request results; {e}")

if __name__ == "__main__":
    listen_for_command()
```

### Installation Requirements:

- pip install SpeechRecognition
- pip install PyAudio

## 4. User Satisfaction Comparison

Efficiency:
- CLI: Most efficient for advanced users.
- GUI: Moderate efficiency.
- VUI: Depends on voice recognition accuracy.

Learning Curve:
- CLI: Steep for beginners.
- GUI: Easy to learn.
- VUI: Very natural but may have recognition issues.

Error Handling:
- CLI: Basic error messages.
- GUI: Visual feedback through pop-ups.
- VUI: Depends on speech clarity.

User Satisfaction:
- CLI: Preferred by technical users.
- GUI: Highly preferred by general users.
- VUI: Convenient but may cause frustration if misinterpreted.

## Conclusion:

CLI provides speed and control for advanced users. GUI offers ease of use and visual clarity for most users. VUI enables hands-free interaction but depends heavily on speech recognition accuracy.